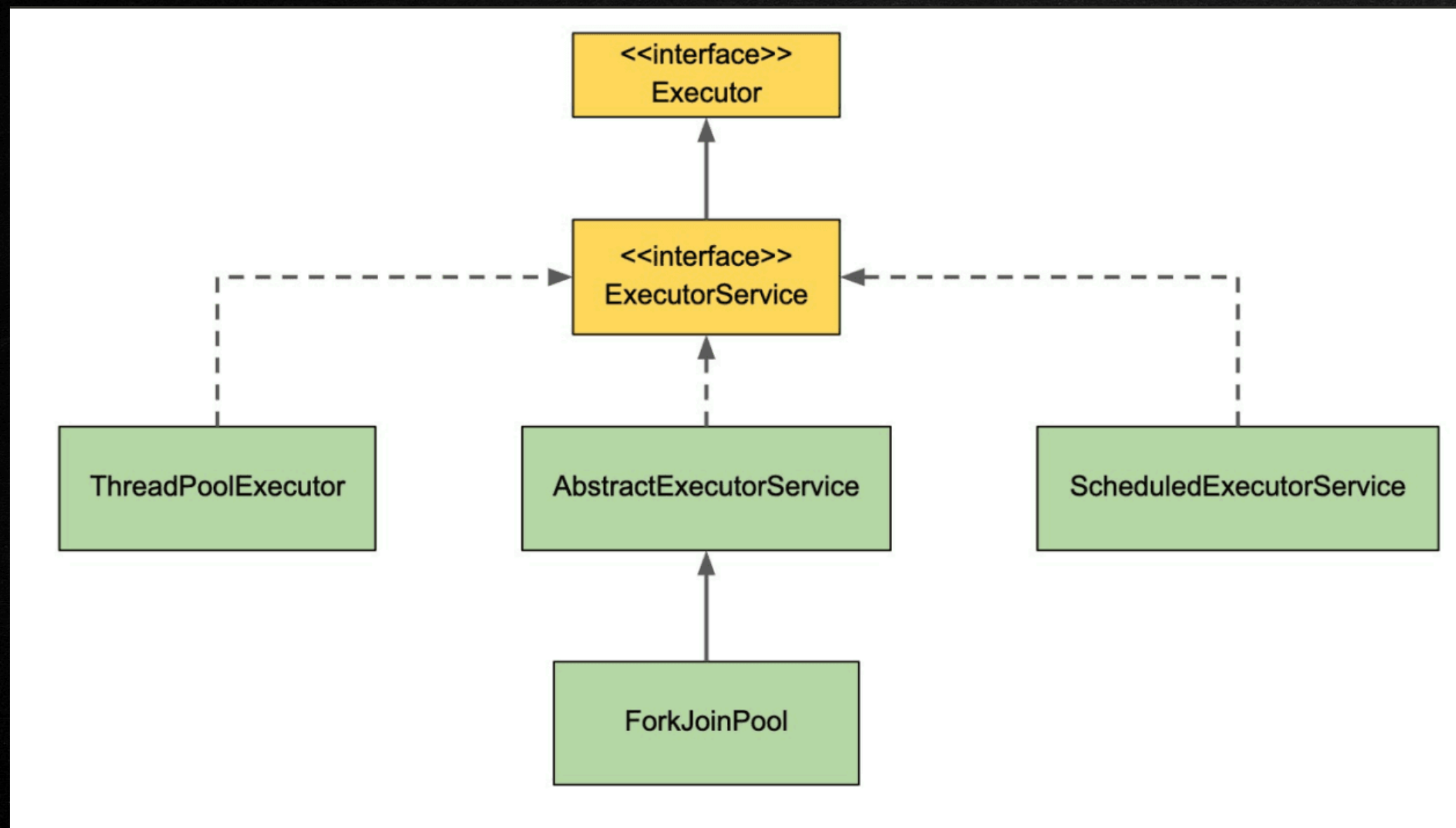


# Executor Service – Recap

Java 5 introduced **java.util.concurrent** – a huge package that solved the thread management problem. The central interface is `ExecutorService`.





# Types of ThreadPoolExecutors

| Factory Method                         | corePoolSize | maxPoolSize                    | Queue            |
|----------------------------------------|--------------|--------------------------------|------------------|
| <code>newFixedThreadPool(n)</code>     | n            | n                              | Unbounded        |
| <code>newCachedThreadPool()</code>     | 0            | <code>Integer.MAX_VALUE</code> | SynchronousQueue |
| <code>newSingleThreadExecutor()</code> | 1            | 1                              | Unbounded        |
| <code>newScheduledThreadPool(n)</code> | n            | <code>Integer.MAX_VALUE</code> | DelayedWorkQueue |

```
// What you write:
ExecutorService pool = Executors.newFixedThreadPool(4);

// What Java actually creates behind the scenes:
ExecutorService pool = new ThreadPoolExecutor(
    4,                          // corePoolSize
    4,                          // maximumPoolSize
    0L, TimeUnit.MILLISECONDS, // keepAliveTime
    new LinkedBlockingQueue<Runnable>() // work queue
);
```



# Future in ExecutorService

When you submit a Callable to an ExecutorService, you get a Future back immediately. The Future is basically a ticket — it represents a value that will exist at some point.

```
Future<String> ticket = pool.submit(() -> {
    Thread.sleep(2000);
    return "result";
});

// ticket is returned IMMEDIATELY — the task is still running
System.out.println("Task submitted, doing other work...");
```

```
future.get()           // blocks — the main flaw
future.get(2, TimeUnit.SECONDS) // blocks with timeout — slightly better
future.isDone()        // polls status — requires a busy-wait loop
future.cancel(true)    // cancels if possible — unreliable
future.isCancelled()   // checks cancel status
```



# The Problem with Future

The fundamental problem: Future has no way to say "when the result is ready, do this next thing." You either block and wait, or you poll in a loop. Neither is good.

You can't write: `future.whenDone(result -> process(result))`

```
// .get() is where you redeem the ticket
// But it BLOCKS the calling thread until the result arrives
String result = ticket.get(); // calling thread pauses here for ~2 seconds
```

```
// Polling - ugly and wasteful
while (!future.isDone()) {
    Thread.sleep(100); // busy-waiting
}
String result = future.get();
```



# CompletableFuture

CompletableFuture implements Future (so it's backwards compatible) but adds the ability to chain operations without blocking. It was introduced in Java 8. You can also do:

- **Chaining & Pipelining:** You can attach callbacks
- **Non-Blocking Operations:** Process results asynchronously without idling the calling thread.
- **Combining Multiple Futures:** You can easily combine independent futures using methods like `thenCombine()` (for two tasks) or `allOf()` / `anyOf()` (for multiple tasks).
- **Robust Exception Handling:** It includes built-in methods like `.exceptionally()`
- **Manual Completion:** Can be completed manually at any time using `.complete(value)` or `.completeExceptionally(ex)`.



# CompletableFuture Methods

```
// thenApply → like map() in streams – transform the value
.thenApply(user -> user.getName())

// thenCompose → like flatMap() – when the next step is also async
.thenCompose(userId -> fetchOrdersAsync(userId))

// thenAccept → consume the result, return void
.thenAccept(result -> saveToCache(result))

// exceptionally → handle errors (like onErrorReturn in Reactor)
.exceptionally(err -> "default value")

// thenCombine → combine two CompletableFutures (like Mono.zip)
CompletableFuture.allOf(future1, future2, future3)
```